Behavioral Stories (Context, Action, Result, Learning)

Story	Context	Action	Result	Learning
Feature Y	At BigCo, we were struggling to close additional clients because our product wasn't different enough from the competition.	I suggested we build Feature Y, but first I explored the technical side with a proof of concept. Once leaders could see it was possible, I worked with design, PM, data science, and a senior engineer to plan and build it over a quarter.	The feature brought in 50% more revenue.	A proof of concept is one of the best ways to convince leaders of what's possible.
Postgres and OS zero-downtime upgrade	Our Trust team required us to fix critical security CVEs by upgrading the legacy PostgreSQL and Ubuntu OS across about 40 production clusters. We were on PostgreSQL 9.6, which has no native logical replication, so an in-place upgrade risked long downtime and costly customer refunds. The breaking changes also touched several Java and infrastructure services.	I designed a two-phase zero-downtime plan. 1. First, I upgraded PostgreSQL from 9.6 to 18.6 with `pg_upgrade`. I added feature flags in the Java and infrastructure services so they worked across both versions without deployment locks, and I built dynamic front-end outage banners to announce the maintenance window. 2. Then I used native logical replication to stream change data capture to a mirrored Ubuntu DB cluster. This avoided the Ubuntu 18.04 libc index invalidation problem and let us cut over with almost no downtime.	I migrated all ~40 production clusters across every deployment stamp. PostgreSQL downtime averaged about 1 hour, and the OS upgrade cutover took about 5 minutes per cluster. Incidents dropped by 40% across replication lag, kernel panics, and I/O saturation. We also got full Trust clearance, which unblocked integration with Microsoft 365 apps such as Teams and Copilot.	High-risk infrastructure upgrades need a solid rollback plan and early executive alignment. I also learned how to build confidence with VP-level leaders and drive consensus across teams when a change has wide impact.
Secret Scanner	Several Microsoft 365 services kept having production outages from expired certificates and secrets, while the Trust org required a strict 60-day rotation. Teams managed secrets in very different ways, often in spreadsheets or local notes, with no central visibility. M365 leadership set a goal of zero certificate- and secret-related outages.	I led the design and rollout of Secret Scanner, a central monitoring platform for M365. 1. **Discovery:** I automated scanning of offline stores (Azure Key Vaults) and live environments (VMs and clusters), and removed duplicate and obsolete credentials. 2. **Alerting:** I built one reporting engine that alerts owners 15 days before expiry. 3. **Adoption:** I wrote onboarding docs, set governance standards, and held weekly office hours so teams could join easily.	All M365 application teams adopted it, and we hit the goal of zero certificate- and secret-related outages. We now track thousands of credentials with automated 15-day alerts, so no one tracks them by hand and we stay compliant with Trust requirements.	Large security programs mean balancing strict compliance with empathy for the teams involved. A technical solution only works across different architectures when it comes with clear standards, easy self-serve onboarding, and steady advocacy.
Search infrastructure	Our search infrastructure ran on Elasticsearch with manual pipelines, so rolling out a patch or creating a cluster was very hard. We were also paying Elastic a license fee just to get TLS for node-to-node security.	I proposed forking the open-source Elasticsearch repo and implementing TLS ourselves, so we wouldn't pay the license fee. I also proposed moving Elasticsearch onto AKS (Azure Kubernetes Service) so we could easily patch, create, and upgrade clusters.	Patching and upgrading a cluster went from days to minutes with one click. We also saved millions of dollars a year in license fees.	I should have brought in the legal team early. Forking and using an open-source project at Microsoft needed their review, and I only realized it late, which delayed the project.

Story	Context	Action	Result	Learning
Database health check issue	The services team reported intermittent read failures that caused micro outages for customers.	I didn't guess. I broke the read path into layers (app, connection pooler, replicas), formed a hypothesis for each, and tested them one by one. I found one replica that was unresponsive because of a disk problem. The pooler's health check didn't test disk availability, so it kept sending reads to that replica. I fixed the health check and set up a process to automatically remove faulty replicas from rotation.	I removed a silent failure mode, improved read availability, and prevented a whole class of similar incidents.	When it's unclear what's wrong, map the system, list hypotheses, and rule them out with data before making changes.
IRIS	On-call toil kept growing in the Infrastructure team, and engineers spent more time on toil than on project work.	I came up with IRIS, a multi-agent AI system that automates infrastructure requests and helps on-call engineers resolve live-site incidents through a chat interface. I added a feedback loop so IRIS could learn from its mistakes and give fewer hallucinated answers. I also worked through the security and trust approval, since Microsoft has no company-wide guidance on AI agents making decisions on live production systems.	IRIS now resolves about 30% of infrastructure requests.	I learned how to reduce AI hallucinations with feedback loops. I also learned that getting security and trust approval for a new kind of system is often the hardest part, especially when no guidelines exist yet.
Maestro adoption	Maestro is an AI agent orchestration tool where multiple agents work together across the whole software development lifecycle. It started at a hack week.	I lead the adoption. I asked 15 service teams, about 150 engineers, to bring Maestro into their development lifecycle, and I put together the process and onboarding documentation.	The tool and onboarding docs are done. 10 teams are fully onboarded, and the rest are still evaluating it for daily work.	The hardest part is understanding every team's pain points and designing a solution that fits everyone's needs. A few teams pushed back on adopting it, and convincing them is the biggest challenge.
Teams Q&A integration	*To fill in.*	*To fill in.*	*To fill in.*	*To fill in.*
Homefeed vs Rankie failure	We had a production issue where Rankie failures caused Homefeed failures.	I proposed that Homefeed shouldn't fail when Rankie is unavailable. It should fail open and show customers a basic feed. The service team disagreed because Rankie's ranking is what makes the feed good for customers. We had several meetings, then involved the product team to decide.	Rankie failures were rare, and building a backup feed was significant work, so the product team put it in the backlog.	Something can be right in theory, but in practice you have to weigh the effort and urgency of the fix.
Kafka consumer retries	Our VentSpout service pulls events from Kafka, pushes them to consumer services, and waits for a 2xx response. When a consumer's downstream service had an outage or network partition, consumers kept failing and VentSpout kept retrying, which saturated its resources.	A teammate proposed exponential back-off instead of constant-delay retries. While reviewing the PR, I suggested adding jitter, a random delay per consumer, so retries wouldn't all line up and would spread out over time.	With jitter in place, VentSpout no longer ran short of resources during these failures.	When I review a PR, I ask whether this is the best approach and whether there's something better, not just whether the code is correct.
Embeddings on separate cluster	*To fill in.*	*To fill in.*	*To fill in.*	*To fill in.*

Story	Context	Action	Result	Learning
HDFS retirement vs CosmosDB cost	*To fill in.*	*To fill in.*	*To fill in.*	*To fill in.*
AI suggestion of Kafka offset commit change	*To fill in.*	*To fill in.*	*To fill in.*	*To fill in.*
FaultLine	*To fill in.*	*To fill in.*	*To fill in.*	*To fill in.*